



Essay / Assignment Title:

Development of an End-to-End NLP Pipeline for Customer Feedback Intelligence and Sentiment Driven Decision Support

Programme title:

Natural Language Processing

Name:

Vidhu

Year:

2026

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters):

VIDHU

.....

Date: 01/10/2026

CONTENTS

1. Introduction
 2. Dataset Description
 3. Task 1: Text Preprocessing and Exploratory Analysis
 4. Task 2: Classical NLP Baseline
 5. Task 3: Semantic Embeddings and Similarity Analysis
 6. Task 4: Transformer-Based Classification
 7. Task 5: Responsible NLP and Emerging Trends
 8. Conclusion
- References

1. INTRODUCTION

Nowdays, big giants like KPMG, Meta, Amazon, Google etc. received lot of reviews and complaints in day to day basis about their products in markets, these are really necessary for organizations to see what is wrong and what is good with their product, so these organizations don't try to review and classify these reviews manually. Instead of doing all the process manually they used NLP pipeline to solve this major problem to analyze and automate the work easily. NLP stands for Natural Language Processing and its pipeline includes text preprocessing, exploratory analysis, TF-IDF classification, semantic embeddings, transformer-based classification and responsible NLP analysis which helps machines to understand, interpret the text by converting it to tokens and by automating the analysis of overall data of customer feedback, it get train to recognize patterns and sentiment analysis, enabling efficient categorization of unstructured data. In this report we are going to take a sample unstructured dataset and process it with NLP pipeline and techniques to train are model to develop and evaluate an end-to-end NLP pipeline for analysing customer feedback and supporting sentiment-driven decision making.

2. Dataset and Experimental Methodology

2.1. Dataset Description

The dataset name is Consumer Complaints Dataset for NLP, it is directly downloaded from Kaggle, this data is collected during march 2020 to march 2021 by the Consumer Financial Protection Bureau (CFPB) which is a federal U.S. agency that acts as a mediator when disputes arise between financial institutions and consumers. Via a web form, consumers can send the agency a narrative of their dispute. This dataset contains three columns which are Ticket Id, Text and category.

- Ticket Id is a unique number given to each complaint.
- Text is complaint the user have also this column is our independent variable.
- Category is a type of column which shows each complaint lies in which category, as of now there are five categories credit reporting, debt collection ,mortgages and loans (includes car loans, payday loans, student loans, etc.), credit cards and retail banking (includes checking/savings accounts, as well as money transfers, Venmo, etc.). This category column is also our target column or dependent variable.

Complaint Classification Dataset				
	ticket_id		text	category
0	0	purchase order day shipping amount receive product week sent followup email exact verbiage paid two day shipping received order company ...		credit_card
1	1	forwarded message date tue subject please investigate comenity bank retailer card scam sent hello name scammed comenity bank credit card...		credit_card
2	2	forwarded message cc sent friday pdt subject final legal payment well fargo well fargo clearly wrong need look actually opened account s...		retail_banking
3	3	payment history missing credit report specialized loan servicing sl made mistake put account forbearance without authorization knowledge...		credit_reporting
4	4	payment history missing credit report made mistake put account forbearance without authorization knowledge matter fact automatic payment...		credit_reporting
5	5	payment history missing credit report made mistake put account forbearance without authorization knowledge matter fact automatic payment...		credit_reporting
6	6	va date complaint experian credit bureau involved decision maker stop cease retaliation discrimination abuse american taxpayer today exp...		credit_reporting
7	7	account reported abbreviated name full name servicer account transunion reporting date last payment reporting payment due account time h...		credit_reporting
8	8	account reported abbreviated name full name servicer account experian reporting date last payment reporting payment due account time how...		credit_reporting
9	9	usdoexxxx account reported abbreviated name full name servicer account reporting date last payment equifax reporting payment due account...		credit_reporting

Fig.1

After checking all the matrices and all the necessary components, this dataset consists of around 162,421 consumer submissions with zero missing values in each columns and no duplicity found in the overall dataset.

```
(162421, 3)
Column names:
['ticket_id', 'text', 'category']
Data types:
Data Type
ticket_id    object
text         object
category     object
Missing values:
Missing Values
ticket_id    0
text         0
category     0
Number of duplicate rows: 0
```

	ticket_id	text	category
count	162421	162421	162421
unique	162421	124473	5
top	162420	victim identity notified collection creditor several time account belong way received good service company provided police report ftc id...	credit_reporting
freq	1	739	91179

Fig 2

After plotting the dataset we get to know the dataset is totally imbalanced and bends towards credit reporting class with 56%, and the remainder roughly equally distributed (between 8% and 14%) among the remaining classes.

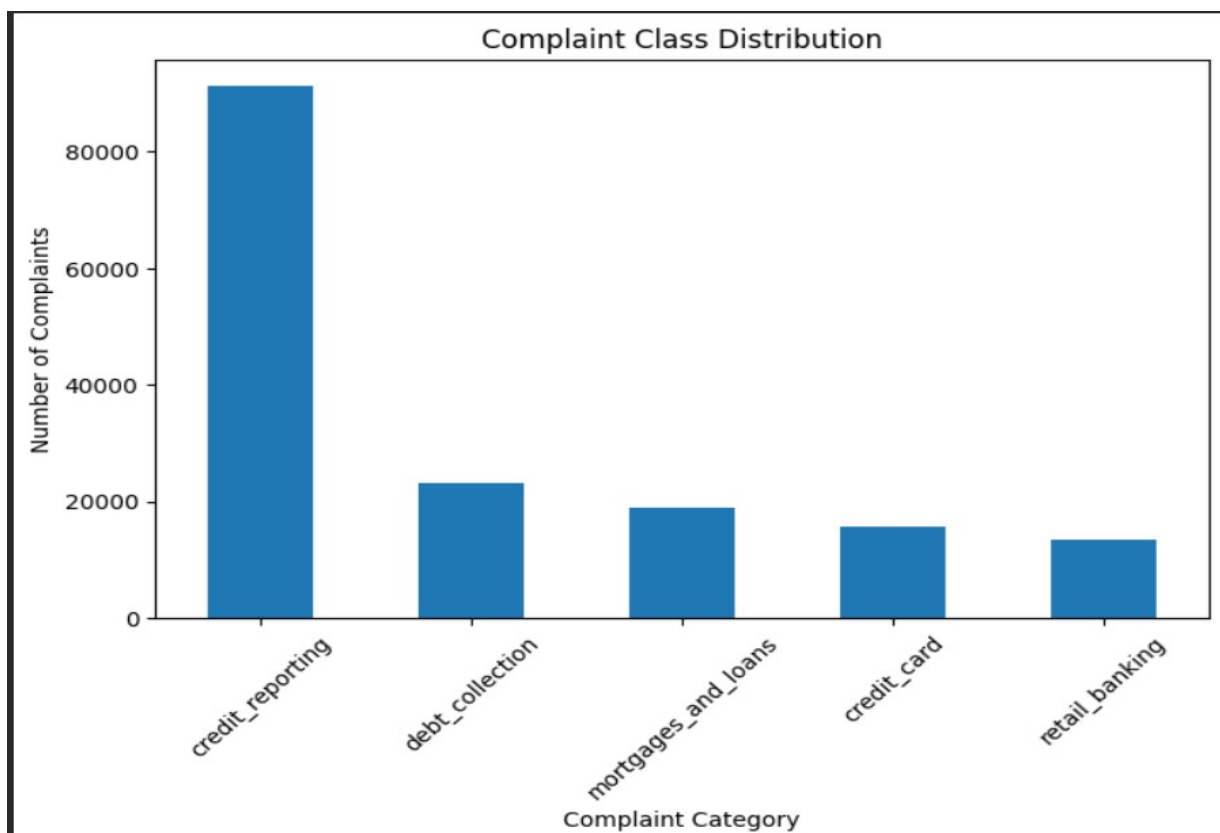


Fig 3

2.2. NLP Pipeline and Experimental Setup

Before starting we need to setup google colab environment to proceed ahead with coding part, here in google colab we can setup the environment use its resources such as GPU's and TPU's free of cost. After setting up the colab environment we proceed ahead with importing necessary libraries.

To complete the NLP pipeline process and prepare for all the task we need necessary pre-requisite library imports from python, scikit learn etc.

Python:- A Python library is a collection of pre-written code and reusable modules that developers use to streamline programming and avoid writing code from scratch. Python's ecosystem includes both a built-in Standard Library (like math or os) and over 900,000 third-party packages hosted on the Python Package Index (PyPI).

Scikit learn:- Scikit-learn (also known as sklearn) is the industry-standard, free, and open-source machine learning library for Python. It is built directly on top of fundamental scientific libraries like NumPy, SciPy, and Matplotlib. Designed for predictive data analysis, it provides a clean and highly unified API across a massive variety of algorithms. This means the code you write to train a model remains virtually identical whether you are predicting stock prices or identifying spam emails.

Apart from these two we need our model to process text and to transform it tokens and all that we require NLTK and spaCy, Gensim and Hugging Face Transformers.

NLTK:- NLTK (Natural Language Toolkit) is the foundational, free, and open-source Python library for Natural Language Processing (NLP). While scikit-learn is built for general machine learning on numbers and vectors, NLTK is built specifically for working with human language data (text). It serves as a massive toolbox filled with text-processing libraries and over 50 corpora and lexical resources (like WordNet).

spaCy:- spaCy is an open-source Python library designed for advanced Natural Language Processing (NLP). Unlike libraries meant for academic research (like NLTK), spaCy is optimized specifically for industrial and production use to process large volumes of text efficiently. It is written in highly optimized Cython and maintained by the software company Explosion.

Gensim:- Gensim is a free, open-source Python library used for unsupervised topic modeling, document indexing, and similarity retrieval. It is highly popular in the natural language processing (NLP) and information retrieval communities for its ability to analyze raw, unstructured text to find underlying statistical patterns.

Hugging Face Transformers:- Hugging Face Transformers is an open-source deep learning library. It provides pretrained models for natural language processing (NLP), computer vision, audio, and multimodal tasks. Built by Hugging Face, it serves as a central hub for machine

learning developers to download, train, and deploy massive transformer architectures (like BERT, GPT, T5, and ViT).

```
1  import re
2  import random
3  import math
4  import time
5  from collections import Counter
6
7
8  import numpy as np
9  import pandas as pd
10 import matplotlib.pyplot as plt
11
12
13 from sklearn.feature_extraction.text import TfidfVectorizer
14 from sklearn.linear_model import LogisticRegression
15 from sklearn.pipeline import Pipeline
16 from sklearn.utils import resample
17 from sklearn.model_selection import train_test_split
18 from sklearn.metrics import (
19     accuracy_score,
20     precision_recall_fscore_support,
21     classification_report,
22     confusion_matrix,
23     ConfusionMatrixDisplay
24 )
25 from sklearn.metrics.pairwise import cosine_similarity
26
```

Fig 4

```
#Helping functions necessary library
import nltk
import spacy
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from spacy.lang.en.stop_words import STOP_WORDS
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')
nltk.download('punkt_tab')

#Semantic Embeddings
!pip install gensim
from gensim.models import Word2Vec

#PCA visualization
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

#Fine tuning pre-requisite
!pip install -q transformers datasets accelerate

from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    DataCollatorWithPadding,
    TrainingArguments,
    Trainer
)
```

Fig 5

3. Task 1 – Text Preprocessing and Exploratory Analysis

3.1. Data Inspection and Cleaning

During inspection of dataset as we already discussed in 2.1 Dataset Description we get to know that total number of records are 162,421 out of which is 56% of data is about credit reporting due to which this dataset is highly leaned toward only one class and other 4 classes have 8 to 14% of data equally(For reference check fig 1 and fig 3). But the good thing with the dataset is that it doesn't have duplicate complaint records as well as no missing value found(For reference check fig 2).

Now after inspecting we can move on cleaning our raw text for further usage in cleaning part we remove double space to single space and special characters for this we create a helper function to clean our raw text with the help of Regex.

```
#Cleaning Raw Text
def clean_raw_text(text):
    text = text.lower()
    text = re.sub(r"^[^a-z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()

    return text
```

Fig 6

3.2. Tokenisation, Stop Words and Lemmatization

Tokenisation, stop words and lemmatization is the very crucial steps before training our model as our model cannot be trained on cleaned or raw text.

Tokenisation:- Tokenization is the fundamental, foundational step in any Natural Language Processing (NLP) pipeline. It is the process of breaking down raw, unstructured text into smaller, manageable units called tokens. Because computers do not inherently understand human language, raw strings must be mapped into structured, discrete components before they can be converted into numerical values (IDs or vector embeddings) that machine learning algorithms can process.

Stop Words:- Stop words are high-frequency, low-meaning words in a language that are commonly filtered out during the preprocessing phase of Natural Language Processing (NLP) pipelines. These include articles, conjunctions, and prepositions like "the", "is", "in", "and", and "a". Because they appear in almost every context, they carry very little unique semantic value or informational signals regarding the core topic of a text.

Lemmatization:- Lemmatization is a foundational text preprocessing technique in Natural Language Processing (NLP) that reduces words to their dictionary base form, which is known as

lemma. Lemmatization relies on linguistic knowledge, grammar rules, and context to ensure that the resulting word is a valid entry in a standard lexicon. For instance, the words "running", "runs", and "ran" are all mapped back to the lemma "run". Similarly, irregular transformations like "better" or "best" map to "good".

As we already know these three steps can be done by both NLTK and spaCy so for the instant we create helper function for stop words, tokenization and lemmatization with NLTK and spaCy, to check whether NLTK or spaCy techniques perform better with our dataset.

```
#Creating Tokens with nltk
def create_tokens_with_nltk(text):
    text = clean_raw_text(text)
    tokens = word_tokenize(text)

    return tokens

#Creating Tokens with spacy
def create_tokens_with_spacy(text):
    text = clean_raw_text(text)
    text_sp = nlp(text)
    tokens = [token.text for token in text_sp]

    return tokens

#Negated words
negated_words = set(["no", "not", "never", "non", "none", "neither", "nothing", "nobody", "no one", "nowhere", "hardly", "barely", "scarcely", "without", "cannot",
                    "can't", "won't", "don't", "doesn't", "didn't", "isn't", "aren't", "wasn't", "weren't", "hasn't", "haven't", "hadn't", "shouldn't",
                    "wouldn't", "couldn't", "mustn't", "needn't", "mightn't", "shan't"])

#Removing Connecting Words with nltk
stop_words = set(stopwords.words('english'))
stop_words = stop_words - negated_words

def remove_connectors_with_nltk(tokens):
    tokens = [token for token in tokens if token not in stop_words]

    return tokens
```

Fig 7

```
#Removing Connecting Words with spacy
stop_words = STOP_WORDS - negated_words

def remove_connectors_with_spacy(tokens):
    return [token for token in tokens if token.lower() not in stop_words]

#Lemmatizing the tokens with nltk
lemmatizer = WordNetLemmatizer()
def lemmatize_with_nltk(tokens):
    lemma_table = pd.DataFrame({
        'Word': tokens,
        'Lemma': [lemmatizer.lemmatize(token) for token in tokens],
        'Lemma as verb': [lemmatizer.lemmatize(token, pos='v') for token in tokens]
    })

    return display_table(lemma_table, title='Lemmatizing the tokens with nltk')

#Lemmatizing the tokens with spacy
def lemmatize_with_spacy(text):
    text_sp = nlp(text)
    lemma_table = pd.DataFrame({
        "Word": [text.text for text in text_sp],
        "Lemma": [text.lemma_ for text in text_sp],
    })

    return display_table(lemma_table, title = 'Lemmatizing the tokens with spacy')
```

Fig 8

3.3. Named Entity Recognition

Named Entity Recognition (NER) is a core natural language processing task that automatically identifies and classifies key elements like people, organizations, and locations within unstructured text.

```
#Entity Recognition
def entity_recognition(text):
    text_sp = nlp(text)
    entities = [{"Entity": ent.text, "Label": ent.label_} for ent in text_sp.ents]

    return display_table(pd.DataFrame(entities), title = 'Entity Recognition')
```

Fig 9

Entity Recognition		
	Entity	Label
0	order day	DATE
1	two day	DATE
2	several week	DATE
3	first	ORDINAL
4	first	ORDINAL
5	mon	PERSON

Fig 10

3.4. NLTK vs spaCy Comparison

After creating all the helper functions with NLTK and spaCy we processed our dataset with both the functions and check whether which suits for our dataset.

Preprocessing using NLTK

```
Preprocessing using nltk

Token Counter Counter({'order': 13, 'purchase': 5, 'shipping': 5, 'purchased': 5, 'amount': 4, 'canceled': 4,
Vocabulary size: 144

Lemmatizing the tokens with nltk
  Word      Lemma  Lemma as verb
0  purchase  purchase  purchase
1    order   order   order
2    day     day     day
3  shipping  shipping  ship
4   receive  receive  receive
...         ...      ...
199  knew    knew     know
200   case   case     case
201  manager  manager  manager
202   ruling  ruling    rule
203   favor  favor     favor
204 rows x 3 columns
```

Fig 11

Preprocessing using spaCy

```
Preprocessing using spacy

Token Counter Counter({'order': 13, 'purchase': 5, 'shipping': 5, 'purchased': 5, 'amount': 4,
Vocabulary size: 146

Lemmatizing the tokens with spacy
  Word      Lemma
0  purchase  purchase
1    order   order
2    day     day
3  shipping  shipping
4   amount  amount
...         ...
227 anything  anything
228   case    case
229  manager  manager
230   ruling  ruling
231   favor  favor
232 rows x 2 columns
```

Fig 12

Comparison table for nltk and spacy

Comparison table for nltk and spacy			
	Feature	NLTK	spaCy
0	Token Count	204	204
1	Vocabulary Size	144	146
2	NER	Not built-in	Built-in
3	Lemmatization	WordNetLemmatizer	Context-aware
4	Speed	Slower	Faster
5	Pipeline	Educational	Production

Fig 13

We come to conclusion after foreseeing that NLTK will fit our educational need for preprocessing our dataset for further training.

4. Task 2 – Classical NLP Baseline

As we selected NLTK in our previous check, now we preprocess our dataset with NLTK helper functions to proceed ahead with splitting our dataset for training and testing purpose, 70% for training and 30% for testing.

```
#Pre-Processing the data before training
df_copy["text_after_preprocessing"] = (
    df_copy["text"]
    .apply(clean_raw_text)
    .apply(create_tokens_with_nltk)
    .apply(remove_connectors_with_nltk)
    .apply(lemmatize_tokens)
    .apply(lambda tokens: " ".join(tokens))
)

#Splitting the data
X_Train, X_Test, y_Train, y_Test = train_test_split(
    df_copy["text_after_preprocessing"],
    df_copy['category'],
    test_size = 0.30,
    random_state = 42,
    stratify = df_copy['category'])
```

Fig 14

After splitting we usually train our model but earlier we faced a problem with our dataset that it is unbalanced and leaned towards one class which is credit reporting so to solve that problem we undersample each class in our dataset to balance all the classes.

Downsample:- Downsampling (also known as undersampling) is a data preprocessing technique that rectifies a class imbalance by randomly removing data points from the majority class until its sample size matches the size of the minority class.

Here with our dataset we do the same thing check the size of minority class and then resample our imbalance class to the size of minority class so that model will treat every class equally. After balancing all the classes we will shuffle the dataset for more randomization.

```

# Combine training text and labels
train_data = pd.DataFrame({
    "text": X_Train.values,
    "category": y_Train.values
})

# Find the size of the smallest class
min_class_size = train_data["category"].value_counts().min()

# Undersample each class to the smallest class size
balanced_train_data = pd.concat([
    resample(
        df_copy[df_copy["category"] == category],
        replace=False,
        n_samples=min_class_size,
        random_state=42
    )
    for category in train_data["category"].unique()
])

# Shuffle the balanced data
balanced_train_data = balanced_train_data.sample(
    frac=1,
    random_state=42
).reset_index(drop=True)

X_Train = balanced_train_data["text"]
y_Train = balanced_train_data["category"]

print("Class Distribution AFTER Balancing:")
display(y_Train.value_counts())

```

Fig 15

```

Class Distribution AFTER Balancing:

```

	count
category	
credit_reporting	9475
mortgages_and_loans	9475
retail_banking	9475
credit_card	9475
debt_collection	9475

```

dtype: int64

```

Fig 16

As we see in fig 16 now all the class have the same number of data points and it is not influenced by any other class so now our training data is ready for training our model.

One major thing is that we balance our dataset after splitting because we don't want to alter our main dataset and also there should be more unseen datapoint in testing phase as well.

4.1. NLP Pipeline

Now start with NLP pipeline here are two main parts that we need to focus on first is converting text to numerical features and second one is using appropriate classification model on which we should train, there are three models, Logistic Regression, Linear SVM and Naive Bayes.

TF-IDF Vectorization:- TF-IDF (Term Frequency-Inverse Document Frequency) vectorization is a statistical method used in natural language processing (NLP) to turn text into numbers by measuring how important a word is to a document in a larger collection.

Logistic Regression:- Logistic regression is a fundamental supervised machine learning algorithm used in Natural Language Processing (NLP) to predict the probability that a given piece of text belongs to a predefined category.

Linear SVM:- Linear Support Vector Machines (Linear SVM) are powerful supervised machine learning models that find an optimal decision boundary (hyperplane) to separate text or token classes in high-dimensional feature spaces.

Naive Bayes:- Naive Bayes is a foundational, probabilistic machine learning algorithm widely used in Natural Language Processing (NLP) for text classification. It calculates the probability of a given text document belonging to a specific class (such as "Spam" or "Not Spam") based on the words it contains, selecting the class with the highest probability.

```
1  #Model Pipeline
2  LR_model = Pipeline([
3      ('vectorizer', TfidfVectorizer(ngram_range = (1,2))),
4      ('classifier', LogisticRegression(class_weight="balanced",max_iter=1000))
5  ])
6
7  #Training the model
8  LR_model.fit(X_Train, y_Train)
9
10 #Predicting the results
11 y_pred = LR_model.predict(X_Test)
12
13
14 #Evaluation metrics
15 print('Classification Report: ')
16 print(classification_report(y_Test, y_pred, zero_division=0))
17 print('\n')
18 print('Confusion Matrix: ')
19 print(confusion_matrix(y_Test, y_pred))
20 print('\n')
21 print("Accuracy score: ")
22 print("Accuracy:", accuracy_score(y_Test, y_pred))
```

Fig 17

As per our model we use TF-IDF Vectorization and Logistic regression in ou model pipeline to train. Now lets take a look over the classification report on which parameters our model train on.

```

Classification Report:

```

	precision	recall	f1-score	support
credit_card	0.71	0.81	0.76	4670
credit_reporting	0.96	0.81	0.88	27354
debt_collection	0.68	0.83	0.75	6945
mortgages_and_loans	0.73	0.90	0.80	5697
retail_banking	0.79	0.93	0.85	4061
accuracy			0.83	48727
macro avg	0.77	0.86	0.81	48727
weighted avg	0.85	0.83	0.84	48727

```

Confusion Matrix:
[[ 3798  214  113  115  430]
 [ 1006 22229 2411 1421  287]
 [  230  542 5736  323  114]
 [  104  196  130 5127  140]
 [  203   16   25   60 3757]]

Accuracy score:
Accuracy: 0.8341781763703902

```

Fig 18

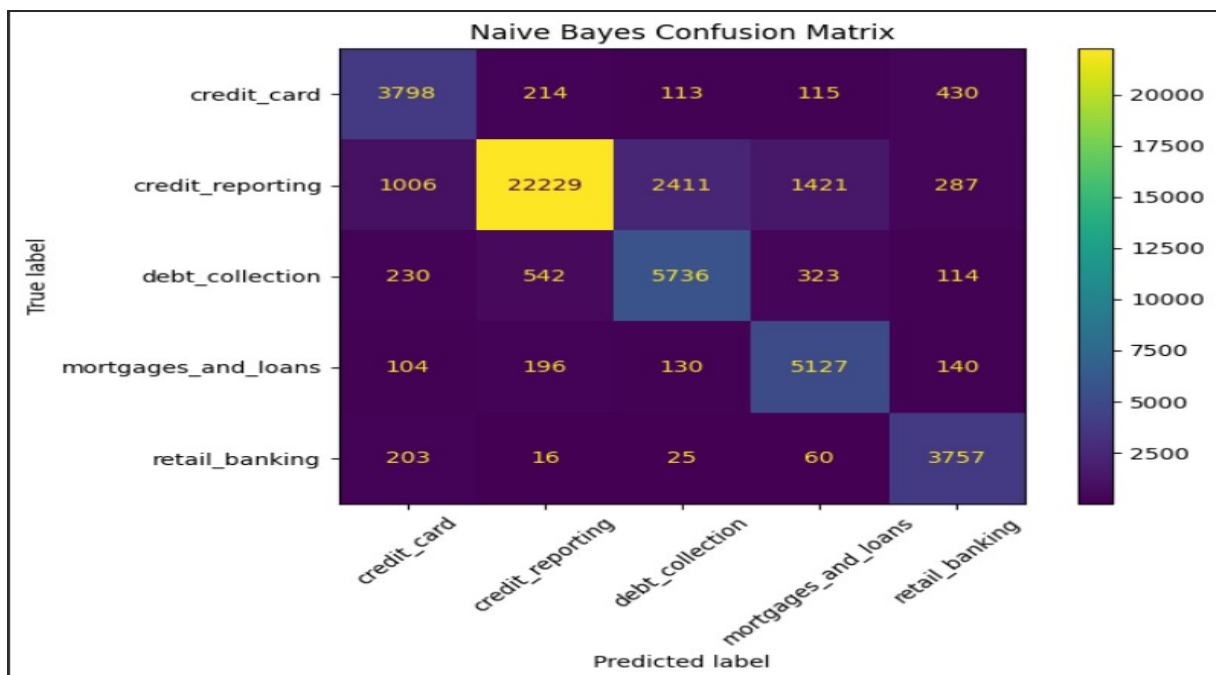


Fig 19

As we see our model accuracy is round 83.4% and all the matrices such as precision, recall, fl score and support in fig 18 for each class, which shows quite good performance for our model.

Let take a look on error analysis table as well to get to know more about our model where it is predicting accurately with how much percentage.

Prediction-level error analysis:

	Text	True category	Predicted category	confidence	Correct
0	husband direct pay account loan company automatically withdraw account month far half year problem loan company u year account loan comp...	mortgages_and_loans	mortgages_and_loans	0.897627	True
1	pay debt directly hospital never cmre report minor son bill credit report attach receipt pay hospital account cmre account	debt_collection	debt_collection	0.890529	True
2	shellpoint mortgage company mortgage forbearance hight pandemic without permission raise mortgage rate month without inform verbal communi...	credit_reporting	mortgages_and_loans	0.959132	False
3	car loan pay early make large payment month addition monthly payment experian credit report late payment report incorrect reach experian...	credit_reporting	credit_reporting	0.996069	True
4	check credit report notice midwest recovery credit report obtain number conduct research obtain address follow mo dispute debt past appe...	credit_reporting	debt_collection	0.821699	False
...	...	...	...	...	...
48722	process home need pay debt deal escrow company send payment debtor loan process debtor dci owe old speak dci payment arrangement settle ...	debt_collection	mortgages_and_loans	0.586832	False
48723	car business report late never miss payment begin remove late payment account	mortgages_and_loans	credit_reporting	0.491743	False
48724	american express continue break fdcpa law establish debt collection balance past american express call cell phone home phone sunday lega...	debt_collection	debt_collection	0.729761	True
48725	send ftc experian non account xxxdispute xxxdispute xxxdispute xxxdispute xxxdispute xxxdispute xxxdispute xxxdispute	credit_reporting	credit_reporting	0.788589	True
48726	escrow refund date freedom mortgage concern receive escrow refund contact freedom mortgage customer service phone email acknowledgment r...	mortgages_and_loans	mortgages_and_loans	0.893525	True

48727 rows x 5 columns

Fig 20

5. Task 3 – Semantic Embeddings and Similarity

5.1. Embedding Approach and Cosine Similarity

Customer feedback systems often rely on semantic similarity to identify related complaints, retrieve similar reviews, group recurring issues, and support retrieval-based decision support. Now using Gensim Word2Vec we will train our model with the same cleaned data, also we can use sentence transformers but we have a lot of data points available in our dataset we stick with Gensim Word2Vec for now.

Word2Vec:- Word2Vec is a foundational framework in Natural Language Processing (NLP) used to generate word embeddings. It uses shallow, two-layer neural networks to reconstruct the linguistic context of words. Instead of treating words as isolated tokens, Word2Vec maps them into a continuous, high-dimensional vector space where words with similar contextual meanings are placed closer together. Unlike traditional methods like one-hot encoding, which create massive, sparse matrices that cannot capture similarity, Word2Vec creates dense vectors (typically 100 to 300 dimensions). This shift enables mathematical operations on language, famously solving linguistic analogies through vector arithmetic, such as:

$\text{Vector}(\text{"King"}) - \text{Vector}(\text{"Man"}) + \text{Vector}(\text{"Woman"}) \sim \text{Vector}(\text{"Queen"})$ [0.6.14, 0.6.15]

Sentence Transformers:- Pretrained sentence transformers are specialized Deep Learning models designed to map entire sentences, paragraphs, or short texts into a dense, fixed-dimensional vector space. In modern Natural Language Processing (NLP), they act as the backbone for semantic understanding, enabling computers to get the meaning of a text rather than relying strictly on keyword matches. While standard transformer models like BERT evaluate text token-by-token (word-by-word), Sentence Transformers—frequently powered by the open-source library sentence-transformers (built on Hugging Face and PyTorch)—are optimized using architectures like Siamese Networks to ensure that texts with similar meanings sit close to each other in the vector space.

```
1 word2vec_model.wv.most_similar("credit", topn=10)

[('report', 0.7562737464904785),
 ('creidt', 0.6497818231582642),
 ('xxxtransunion', 0.621145486831665),
 ('reoports', 0.6170380711555481),
 ('negativley', 0.6135937571525574),
 ('xxxxtotal', 0.613172173500061),
 ('xxxcredit', 0.609966516494751),
 ('aess', 0.6062547564506531),
 ('bueraus', 0.6049818396568298),
 ('definatly', 0.6047028303146362)]
```

Fig 21

Here we check similar top 10 words related to credit by using word2vec model with their cosine similarities. We can also check it individually by using cosine similarity.

```

1 credit_vector = word2vec_model.wv["credit"]
2 report_vector = word2vec_model.wv["report"]
3
4 similarity = cosine_similarity(
5     [credit_vector],
6     [report_vector]
7 )[0][0]
8
9 print("Cosine similarity:", similarity)

```

Cosine similarity: 0.7562737

Fig 22

Like here we match word credit with word report the cosine similarity of these two words together is 75.62%. As well as we can perform cosine similarity not only on words, we can also compare two sentences which look alike same or not and what is their similarity percentage.

```

1 # Comparing two similar complaints using cosine similarity
2
3 complaint_A = "A debt I already paid is still showing on my credit report."
4 complaint_B = "My credit report has not been updated even though I have cleared the debt."
5
6 print("Complaint A:", complaint_A)
7 print("Complaint B:", complaint_B)
8
9 # Preprocess Complaint A
10 tokens_A = create_tokens_with_nltk(
11     clean_raw_text(complaint_A)
12 )
13
14 tokens_A = remove_connectors_with_nltk(tokens_A)
15 tokens_A = lemmatize_tokens(tokens_A)
16
17 # Preprocess Complaint B
18 tokens_B = create_tokens_with_nltk(
19     clean_raw_text(complaint_B)
20 )
21
22 tokens_B = remove_connectors_with_nltk(tokens_B)
23 tokens_B = lemmatize_tokens(tokens_B)
24
25 # Convert to sentence vectors
26 vector_A = get_sentence_vector(tokens_A, word2vec_model)
27 vector_B = get_sentence_vector(tokens_B, word2vec_model)
28
29 # Calculate cosine similarity
30 similarity = cosine_similarity(
31     [vector_A],
32     [vector_B]
33 )[0][0]
34
35 print("Cosine Similarity:", similarity)

```

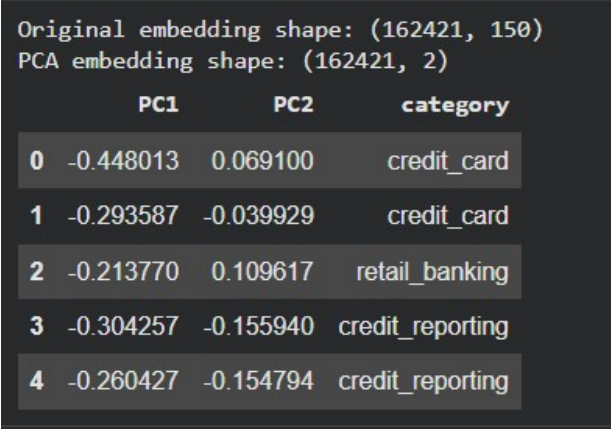
Complaint A: A debt I already paid is still showing on my credit report.
 Complaint B: My credit report has not been updated even though I have cleared the debt.
 Cosine Similarity: 0.8950429

Fig 23

Here in the above figure 23 we show the comparison of two different complaints written by different persons with having same class, these two complaints seen as to be different but the meaning is similar for both of them, that's why the cosine similarity of these two complaints is around 89.5%.

5.2. Embedding Visualisation

PCA Interpretation:- The PCA visualization provides a two-dimensional representation of the customer complaint sentence embeddings. Complaints that appear close to each other in the plot have more similar Word2Vec-based semantic representations, while complaints located farther apart have less similar representations. The separation between complaint categories can be inspected visually. If several categories form partially separated groups, this suggests that the Word2Vec sentence representations capture some category-specific semantic information. However, overlapping categories indicate that different complaint types can use similar vocabulary or discuss related financial issues. PCA is mainly used here as a visualization technique. Reducing the original 150-dimensional vectors to only two dimensions, according to which we lose information, so the visualization should not be interpreted as a complete representation of the original semantic space. The Word2Vec sentence representation also has limitations because each complaint vector is created by averaging individual word vectors. This can lose word order, sentence structure, negation, and contextual meaning. Therefore, semantically different sentences can sometimes obtain similar representations.



```
Original embedding shape: (162421, 150)
PCA embedding shape: (162421, 2)
```

	PC1	PC2	category
0	-0.448013	0.069100	credit_card
1	-0.293587	-0.039929	credit_card
2	-0.213770	0.109617	retail_banking
3	-0.304257	-0.155940	credit_reporting
4	-0.260427	-0.154794	credit_reporting

Fig 24

Here in this figure 24 we can clearly check 150 dimension PC1 converted to 2 dimension PC2 for visualization.

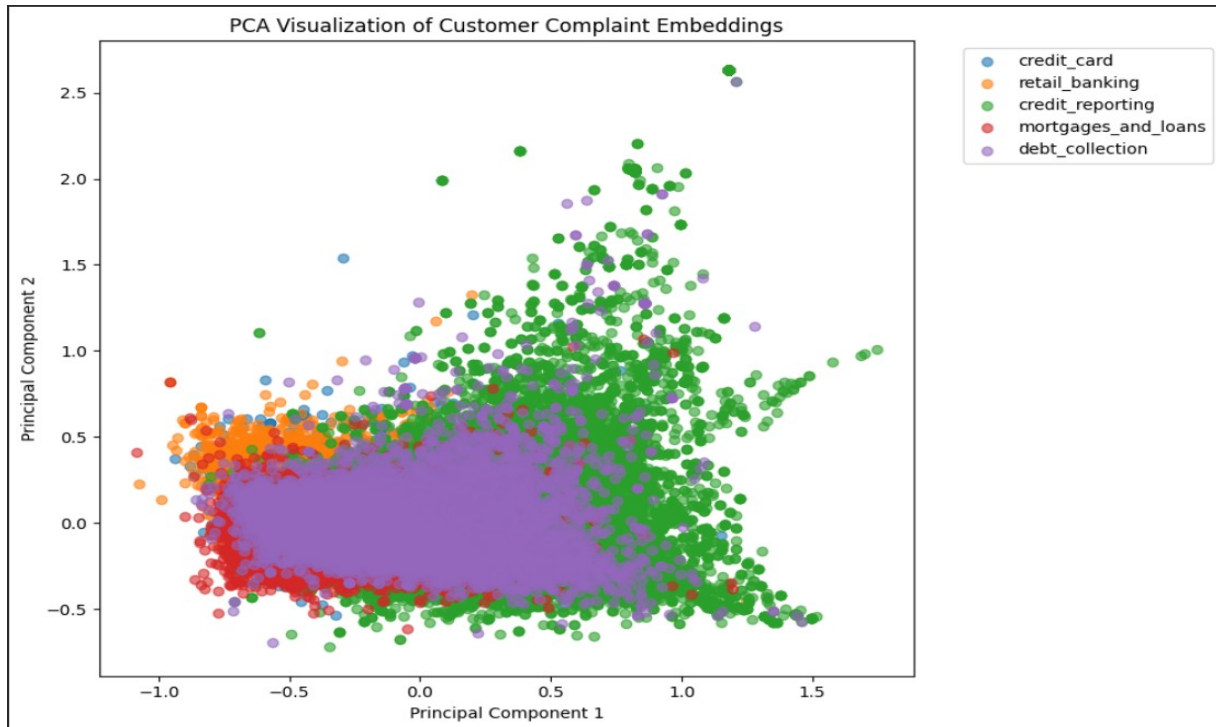


Fig 25

6. Task 4 – Transformer-Based Classification — 450 words

6.1. Transformer Selection

For the transformer stage, we selected DistilBERT. We choose this because it can understand the context of words better than TF-IDF, while being smaller than the original BERT model. The model has 66,957,317 parameters in this implementation.

6.2. Tokenisation and Fine-Tuning

Also for tokenization we use DistilBERT AutoTokenizer to convert each complaint into tokens that the model able to process easily. After removing missing and duplicate records, 4,000 complaints were selected using stratified sampling. The data was divided into 2,800 training examples, 600 validation examples and 600 test examples. Fine-tuning was carried out for 3 epochs with a batch size of 16, learning rate of 2×10^{-5} and weight decay of 0.01. The best model was selected using validation F1.

6.3. Model Evaluation

The test results for DistilBERT are shown below. The model reached 80% accuracy and a weighted F1-score of 79.24%. The macro F1 was 76.95%, which shows that performance was not the same across all complaint classes. The confusion matrix also showed complaints being assigned to different categories. Also DistilBert doesn't produce reproducible results different machines and different GPU or TPU can differ the results in total.

Classification Report - DistilBERT				
	precision	recall	f1-score	support
credit_card	0.77	0.78	0.77	72
credit_reporting	0.83	0.89	0.86	271
debt_collection	0.76	0.57	0.65	102
mortgages_and_loans	0.76	0.74	0.75	90
retail_banking	0.77	0.86	0.81	65
accuracy			0.80	600
macro avg	0.78	0.77	0.77	600
weighted avg	0.79	0.80	0.79	600

Fig 26

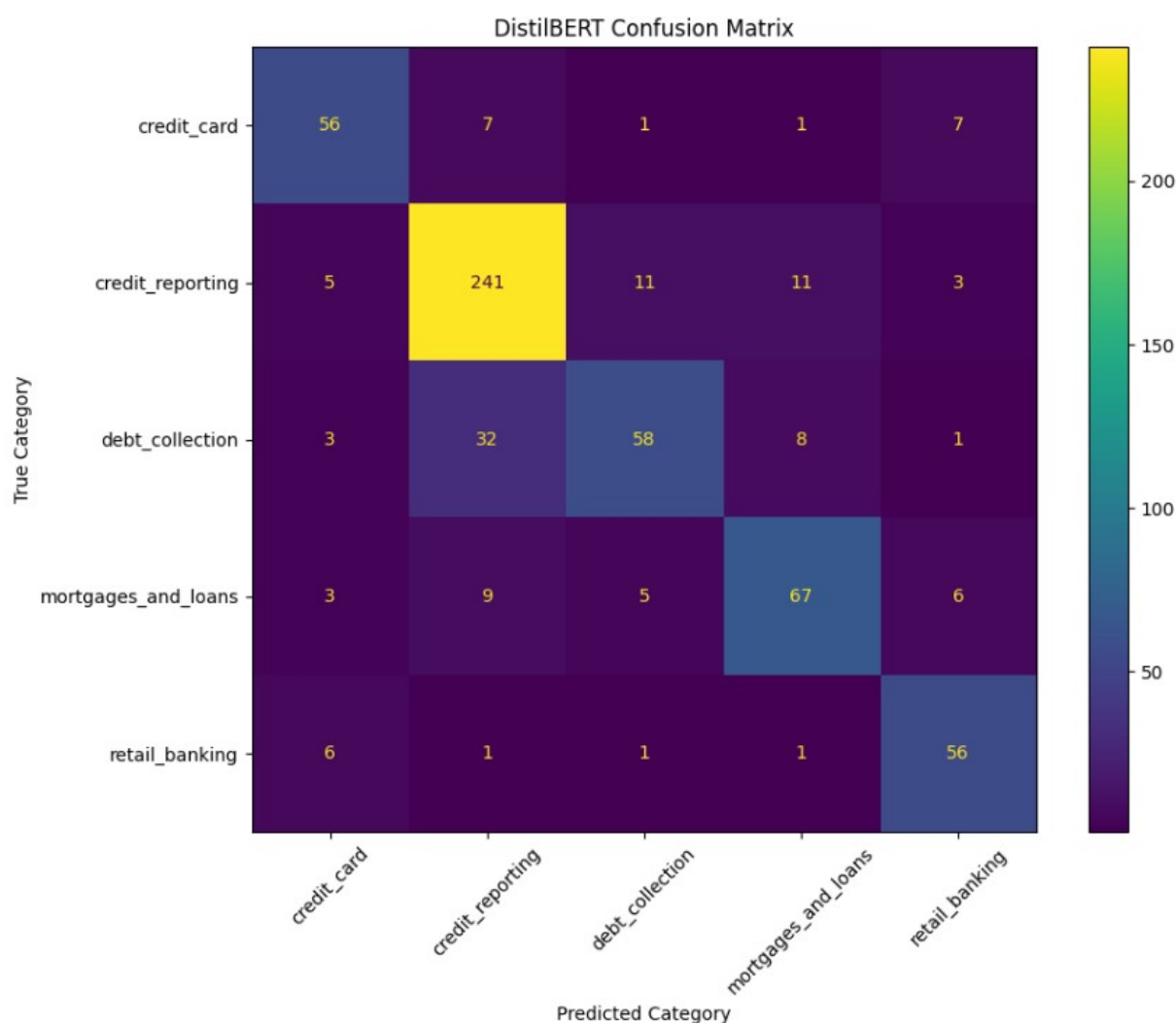


Fig 27

6.4. Classical vs Transformer

The comparison shows that Logistic Regression had slightly higher test scores than DistilBERT. Logistic Regression has 83.42% accuracy and 83.82% weighted F1, while DistilBERT reached 79.67% accuracy and 79.24% weighted F1. However, the two models were not trained on the same number of examples. Logistic Regression used the larger dataset after balancing, while DistilBERT used 4,000 examples. Therefore, the results should be read as an experiment rather than a direct like-for-like test. Logistic Regression is easier to explain because its TF-IDF features can be inspected. In this run, its training time was 166.35 seconds, compared with 203.28 seconds for the Logistic Regression pipeline.

	Model	Accuracy	Precision	Recall	Weighted F1	Macro F1
0	Logistic Regression	0.8342	0.8546	0.8342	0.8382	0.8089
1	DistilBERT	0.7967	0.7945	0.7967	0.7924	0.7695

Fig 28

	Model	Training Time (seconds)
0	Logistic Regression	203.28
1	DistilBERT	166.35

Fig 29

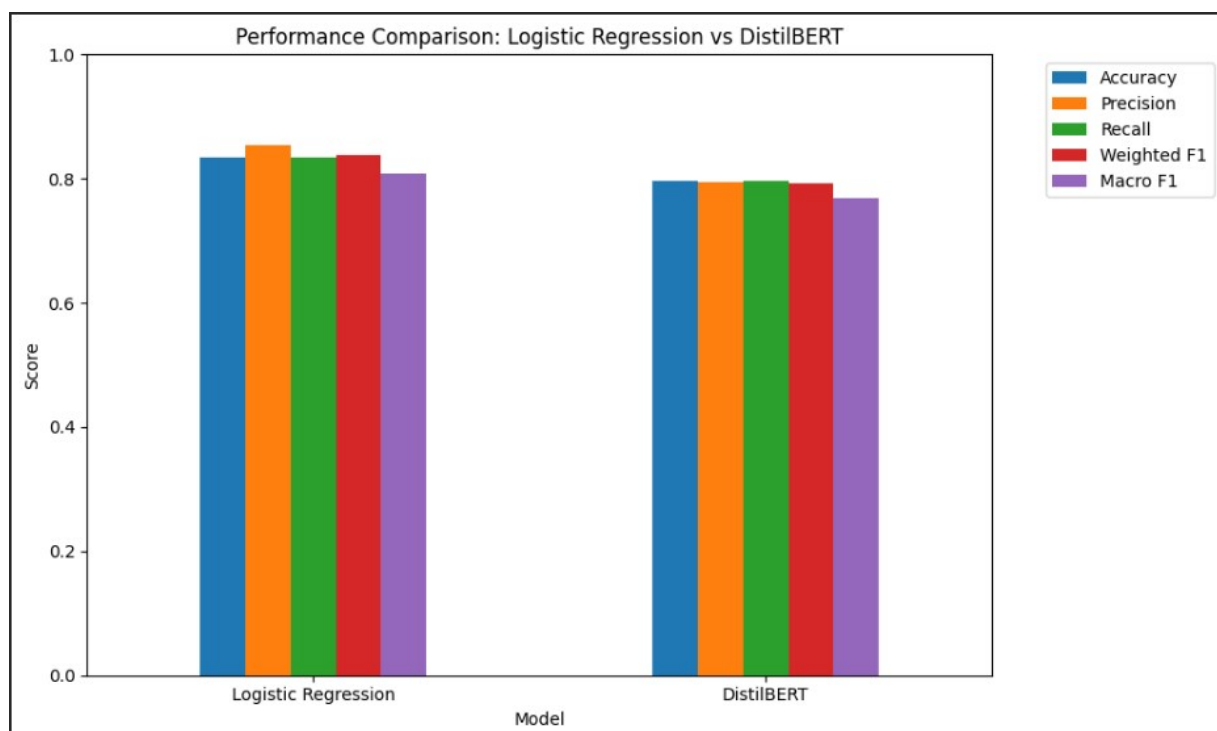


Fig 30

Conclusion

This project gave me an opportunity to apply different NLP techniques to a customer complaint dataset and understand how each method performs in practice. The preprocessing stage using NLTK and spaCy helped prepare the text for further analysis and also showed the importance of making sensible choices about tokenisation, stop words and lemmatisation. The exploratory analysis provided a clearer view of the complaint categories and the language used by customers.

For classification, the TF-IDF based Logistic Regression model produced an accuracy of 83.42% and a weighted F1-score of 83.82%. This provided a useful baseline and showed that traditional machine learning can still perform well when the text is represented appropriately. The Word2Vec experiment also demonstrated the value of semantic representations. The cosine similarity result of 0.93 in the retrieval example suggested that the embedding approach could identify related complaint content.

The transformer experiment gave a different perspective. DistilBERT achieved an accuracy of 79.67% and a weighted F1-score of 79.24% in this experiment. Although its performance was lower than the Logistic Regression baseline, it demonstrated how contextual language representations can be applied to complaint classification. The comparison also made it clear that model choice should not be based only on accuracy, since training requirements, computational resources, interpretability and practical deployment also matter.

Overall, the work showed me that an effective NLP pipeline is not simply about selecting the most advanced model. Data preparation, feature representation, evaluation and understanding model errors are equally important.

References

- Code file:- <https://colab.research.google.com/drive/15VEzfsd3pJAdVe6deBiZAvn4UzWsZjA?usp=sharing>
- Dataset link:- <https://www.kaggle.com/datasets/shashwatwork/consume-complaints-dataset-fo-nlp>
- Python:- <https://docs.python.org/3/library/index.html>
- Scikit-Learn:- <http://scikit-learn.org>
- NLTK:- <https://www.nltk.org/>
- spaCy:- <https://spacy.io/>
- Gensim:- <https://pypi.org/project/gensim/>
- Hugging Face Transformers:- <https://huggingface.co/docs/transformers/en/index>
- Github Repositories:- <https://github.com/VidhuGupta/Development-of-an-End-to-End-NLP-Pipeline-for-Customer-Feedback-Intelligence->
- Use of AI for some sentence transformation is used with help of gemini and chatgpt also little help of gemini has been taken in fine tuning the model in task4.